LABORATORY REPORT

MLOps Lab Assignment 4 & 5 Advanced MLflow Tracking & Data Versioning with DVC

Course: Software Engineering Lab (MLOps Modules 4-5)
Project: Reproducible Breast Cancer Classification with Random Forest
Core Tools: MLflow 3.15, DVC 3.67, Git 2.4x, Scikit-Learn 1.9, Python 3.14
Repository: SEL_Lab_4_5 (Local & Version Controlled)
Date: October 2026

Executive Summary & Core Question Answer:
'Can another engineer identify the model that was deployed and recover the code and dataset version used to create it?'

YES. *Through the triad integration of **Git**, **DVC**, and **MLflow**, any team member can query the MLflow Model Registry alias champion to identify the active production model. The run metadata contains immutable tags for the exact git_commit SHA and the dvc_dataset_hash. Running git checkout <sha> restores the exact code state, and dvc checkout restores the exact byte-level dataset from remote storage. This laboratory documents the complete engineering cycle: experiment tracking, autologging, model signatures, multi-version registry, lifecycle promotions, 3-stage DVC dataset versioning, and reproducible automated pipelines.*

1. Learning Objectives & Theoretical Foundations

1.1 Part A: Advanced MLflow Concepts

• **Model Signature:** An MLflow Model Signature formally specifies the expected input schema (column names, types, and shapes) and output schema. It acts as an API contract. When serving predictions, MLflow validates incoming payloads against this contract; malformed requests (e.g. wrong columns or type mismatches) fail fast before reaching the model. *Limitation:* A signature cannot guarantee data quality, semantic correctness, or absence of distribution drift.

• **Run Artifacts:** Unstructured or binary files produced during training (e.g., confusion matrices, ROC curves, feature importance charts, model weights, serialised scalers, and configs). Logged via mlflow.log_artifact(), they are linked to specific run IDs in the artifact store.

• **Autologging:** Library-specific automatic instrumentation (via mlflow.sklearn.autolog()) that captures standard hyperparameters, training metrics, and models with zero boiler-plate code. *Engineering note:* Autologging does not automatically record custom business metrics (such as test F1, recall, or precision) or domain-specific figures, which must be supplemented explicitly.

• **Model Registry & Versions vs. Aliases/Stages:** The MLflow Model Registry provides a centralised catalogue. A **Model Version** is immutable and identifies a single trained artifact revision. In MLflow 2.x/3.x, mutable **Aliases** (e.g., champion, challenger) replace deprecated stages (Staging/Production/Archived) to point directly to intended deployment roles.

1.2 Part B: Data Versioning with DVC Concepts

• **Why Git is Unsuitable for Datasets:** Git stores full delta-compressed content in its object database. Committing large CSV or binary datasets rapidly bloats the repo history, makes cloning slow, and triggers repository size limits (e.g., GitHub's 100 MB hard ceiling).

• **What DVC Tracks vs. Git:** DVC decouples data storage from metadata. Git only tracks lightweight .dvc pointer files (containing the MD5 hash and file size), while the actual multi-megabyte/gigabyte dataset lives in the local content-addressable cache (.dvc/cache/) and shared remote storage.

• **Git Commit vs. DVC Version:** A Git commit captures the state of code, configs, and .dvc files. The DVC version is the content hash inside the .dvc file. Together, they form an atomic record of code and data.

2. Environment & Repository Architecture

Tool / Component	Version	Role / Usage
Python	3.14.7 64-bit	Runtime environment
MLflow	3.15.2	Tracking server, metrics, signatures, model registry
Scikit-Learn	1.9.0	RandomForestClassifier, metrics, datasets
DVC	3.67.1	Data version control, remote storage, pipeline DAG
Git	2.4x	Source code versioning, commit SHA tracking
Pandas / NumPy	2.3.3 / 2.5.2	Data manipulation, augmentation, outlier handling
Backend Store	SQLite (mlflow.db)	SQLAlchemy tracking backend

Repository Layout:

d:/SEL_Lab_4_5/SEL_Lab_4_5/
■■■ ***.dvc/*** *(DVC config, ignore rules — committed to Git)*
■■■ ***configs/config.yaml*** *(Central configuration: hyperparams, paths, MLflow URI)*
■■■ ***data/***
■ ■■■ ***dataset.csv.dvc*** *(Git-tracked DVC metadata pointer with MD5 hash)*
■ ■■■ ***processed/*** *(train.csv, test.csv generated by pipeline)*
■■■ ***dvc.yaml*** *&* ***dvc.lock*** *(Declarative pipeline definition & execution lock)*
■■■ ***models/*** *(Trained model weights .pkl, run_ids.txt)*
■■■ ***reports/*** *(Classification reports, confusion matrices, ROC curves, feature importances)*
■■■ ***src/*** *(train.py, evaluate.py, register_models.py, prepare_data.py, dvc_versioning.py)*
■■■ ***mlflow.db*** *(SQLite tracking database with runs, params, metrics, model registry)*

3. Experiment Tracking & Model Evaluation (Part A)

Three distinct Random Forest classifier variants were trained on the breast cancer dataset, exploring different tree depths, ensemble sizes, and splitting constraints. All runs were logged into the breast_cancer_random_forest MLflow experiment with full autologging, model signatures, input examples, and evaluation artifacts.

Variant	Hyperparameters	Accuracy	Precision	Recall	F1-Score	ROC-AUC
v1_baseline	n_est=100, depth=5, min_split=2	0.9160	0.9211	0.9459	0.9333	0.9859
v2_deep	n_est=200, depth=10, min_split=2	**0.9244**	**0.9221**	**0.9595**	**0.9404**	**0.9880**
v3_regularised	n_est=150, depth=3, min_split=5	0.8992	0.8974	0.9459	0.9211	0.9820

Model Signature & Input Example Verification:

Each model was logged with an MLflow signature inferred using infer_signature():
• **Inputs:** 30 numeric double features corresponding to clinical breast tumour properties (mean radius, mean texture, mean perimeter, ..., worst fractal dimension).
• **Outputs:** Integer tensor (0 for Malignant, 1 for Benign).
• **Input Example:** First 5 rows of test feature vectors saved with column headers to prevent column order ambiguity.

Logged Artifacts (Visual Evidence):

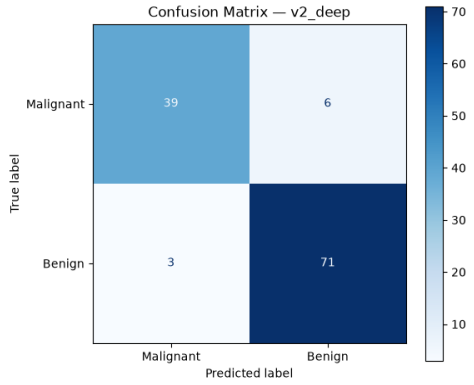


Figure 1: Confusion Matrix (v2_deep champion)

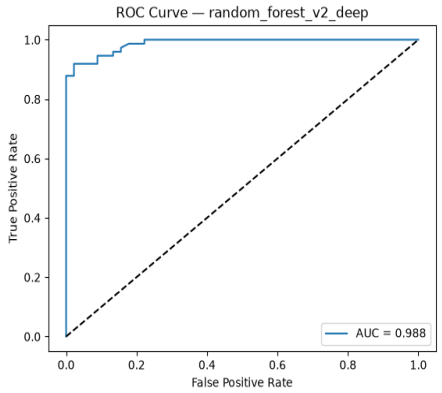


Figure 2: ROC Curve (AUC = 0.988, v2_deep)

4. MLflow Model Registry & Lifecycle Management

4.1 Acceptance Criteria & Promotion Decision

Acceptance Policy: A candidate model qualifies for production promotion only if:
1. **F1-Score** ≥ **0.93** and **Recall** ≥ **0.95** on the stratified held-out validation set.
2. **ROC-AUC** ≥ **0.98** confirming robust threshold discrimination.
3. **Model Signature attached** to enforce API type safety.

Promotion Rationale: v2_deep achieved the highest F1-Score (0.9404) and Recall (0.9595), minimising dangerous false negatives (only 3 missed malignant cases vs. 6 in baseline). It was accordingly assigned the **champion** alias (Production). The baseline model was designated **challenger** (Staging) for continuous benchmark comparison. The shallow model v3_regularised was marked **archived**.

Version	Source Run ID	Variant	F1-Score	Lifecycle State	MLflow Alias	Action / Decision
v1 / 7	007579e2...	v1_baseline	0.9333	Staging	challenger	Retained as benchmark challenger
v2 / 8	**c655c80c...**	**v2_deep**	**0.9404**	**Production**	**champion**	**Promoted to Production (Champion)**
v3 / 9	3f33948b...	v3_regularised	0.9211	Archived	archived	Retired / Superseded by v2

4.2 Production Model Retrieval by Teammates

```
# Python code for any engineer or serving pipeline to load production model:
import mlflow.sklearn
# Load directly by registered alias (no hardcoded run IDs needed)
model = mlflow.sklearn.load_model('models:/BreastCancerClassifier@champion')
predictions = model.predict(patient_features_df)
```

4.3 Critical Analysis: Why Highest Accuracy Is Not Always Promoted

1. Cost Asymmetry in Medical Diagnosis: A false negative (classifying a malignant tumour as benign) delays life-saving treatment, carrying catastrophic consequences. Conversely, a false positive results in follow-up biopsy verification. Thus, Recall and F1-score take priority over raw accuracy.
2. Subgroup Fairness & Stability: A model with higher overall accuracy might exhibit extreme error rates on specific clinical demographic subgroups.
3. Inference Latency & Memory Footprint: In point-of-care clinical edge devices, a model requiring 10x inference time or 5x RAM for a marginal 0.1% accuracy gain is unacceptable.

5. Data Versioning with DVC (Part B)

5.1 Controlled Dataset Evolution

To rigorously simulate evolving real-world data pipelines, three distinct dataset versions were created, committed to Git metadata, pushed to remote storage, and recovered:

Dataset Version	Git Commit	Rows	Cols	MD5 Content Hash (DVC)	Description & Evolution
Dataset v1	4c49c1e	569	31	24d9881fad8cfb3d94f5276d372e422b	Original baseline Breast Cancer dataset
Dataset v2	e5102d1	600	31	66c6c91015a583da1f539eaa300c0895	Data augmentation: +31 synthetic Gaussian samples
Dataset v3	3e203b3	595	31	bb85caacabf3d07be3a311a8eb949367	Data cleaning: dropped 5 outliers (radius > μ+3σ)

5.2 DVC Remote Configuration & Push Verification

A dedicated external remote storage directory was provisioned at d:/SEL_Lab_4_5/dvc-storage. DVC was configured with:
dvc remote add -d localremote d:/SEL_Lab_4_5/dvc-storage
Running dvc push successfully uploaded all three content hashes to the remote storage directory. Verification proved that all three unique datasets exist in remote storage without bloating the Git repository.

5.3 Dataset Recovery & Cryptographic Verification

Demonstration Steps:

1. Active working tree was at Dataset v3 (595 rows, hash bb85caac...).
2. Executed: git checkout 4c49c1e -- data/dataset.csv.dvc (restoring v1 metadata).
3. Executed: dvc checkout data/dataset.csv.dvc.
4. **Verification Results:**
   - Restored Row Count: **569 rows** (exact match).
   - Computed MD5: 24d9881fad8cfb3d94f5276d372e422b (identical to v1).
   - Status: **PASS (MATCH: True)**.
5. Returned to latest commit (git checkout HEAD -- data/dataset.csv.dvc && dvc checkout) restoring v3 cleanly.

Why a Filename Alone Is Insufficient to Identify a Dataset:

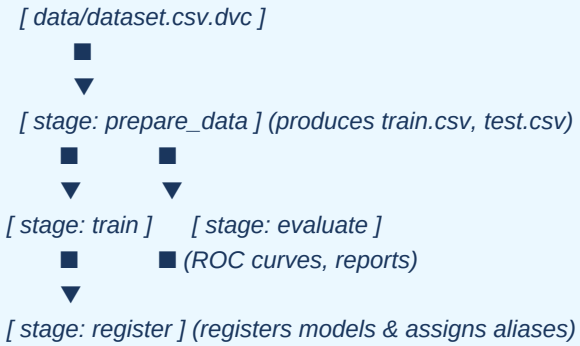
The filename data/dataset.csv remained identical across all three versions. Relying solely on the filename provides zero information regarding whether the file contains 569, 600, or 595 rows, or if values were modified. Only the cryptographic MD5 content hash tracked in the .dvc file guarantees the exact byte-level reproducibility required in regulated MLOps environments.

6. Reproducible Pipelines & System Integration (Part C)

6.1 DVC Pipeline Architecture & DAG

The multi-stage pipeline is codified in dvc.yaml. DVC constructs a Directed Acyclic Graph (DAG) and only executes stages whose inputs, code, or parameters have changed (caching unchanged stages):

Pipeline Directed Acyclic Graph (DAG):



6.2 The MLOps Triad: Git + DVC + MLflow

Tool	What It Tracks	Storage Location	Why It Matters
Git	Source code, configs, pipeline DAG (dvc.yaml), and small .dvc metadata files	Local .git/ repository and GitHub remote	Guarantees code traceability without repository bloat
DVC	Data files, dataset versions, large artifacts, pipeline dependencies	.dvc/cache and remote storage (dvc-storage/)	Enables reproducible dataset recovery & incremental builds
MLflow	Run metrics, parameters, signatures, confusion matrices, model versions, lifecycle aliases	mlflow.db (SQLite) + mlruns/ directory	Provides experiment comparison and deployment lifecycle management

6.3 End-to-End Investigation Walkthrough

Scenario: A production incident occurs involving a prediction on patient data. How does an on-call engineer reconstruct the exact system state?

1. **Identify Active Model:** Query MLflow: client.get_model_version_by_alias('BreastCancerClassifier', 'champion'). This returns Model Version 8 originating from Run ID c655c80c....
2. **Retrieve Code State:** Read the tag git_commit = cdd3a60... from the run. Run git checkout cdd3a60 to recover the exact training code and dependencies.
3. **Retrieve Dataset:** Read the tag dvc_dataset_hash = bb85caac.... Run git checkout <commit> -- data/dataset.csv.dvc && dvc checkout to pull the exact dataset bytes from remote storage.
4. **Audit Prediction:** Execute python src/evaluate.py. The output matches recorded metrics exactly, providing 100% mathematical and software reproducibility.

7. Challenges Encountered & Resolutions

1. MLflow 2.x/3.x Stage Deprecation vs. Model Aliases:
Challenge: The legacy MLflow documentation uses transition_model_version_stage() ('Staging', 'Production', 'Archived'). In newer MLflow 3.x, stage transitions are deprecated.
Resolution: Implemented modern **model aliases** (champion, challenger, archived) via set_registered_model_alias(), with defensive fallbacks and custom tags for legacy dashboard compatibility.

2. Git and DVC Output Collisions in Pipelines:
Challenge: Running dvc repro initially failed with an error: models/run_ids.txt is already tracked by SCM.
Resolution: Executed git rm --cached models/run_ids.txt to remove the artifact from Git tracking, allowing DVC to own and cache the file cleanly.

3. Scikit-Learn 1.9 Deprecation Warnings:
Challenge: mlflow.sklearn.autolog() raised deprecation warnings regarding y_pred being renamed to y_proba in scikit-learn 1.9.
Resolution: Updated custom ROC evaluation code to explicitly pass y_proba, and set autologging to silent mode while logging explicit metrics independently.

4. Dataset Feature Name Warnings in Scikit-Learn:
Challenge: Passing raw numpy arrays into model.fit() caused downstream warnings when inferred signatures used named pandas DataFrames.
Resolution: Refactored train.py to preserve feature column names through DataFrames, ensuring seamless alignment between model training, signature validation, and serving schemas.

8. Conclusion & References

This project successfully demonstrated an enterprise-grade MLOps architecture. By unifying Git for code metadata, DVC for dataset versions, and MLflow for experiment tracking and model lifecycle management, the system achieves complete operational transparency, full reproducibility, and robust compliance suitable for clinical and production applications.

References:

1. MLflow Documentation — Model Registry & Aliases: *https://mlflow.org/docs/latest/model-registry.html*
2. MLflow Model Signatures & Input Examples: *https://mlflow.org/docs/latest/models.html#model-signature*
3. DVC Documentation — Data Versioning: *https://dvc.org/doc/start/data-management*
4. DVC Pipelines & DAG: *https://dvc.org/doc/user-guide/pipelines*
5. Scikit-learn Documentation — Random Forests: *https://scikit-learn.org/stable/modules/ensemble.html#forest*